

Machine Learning

Lecture 7: Deep Learning I

Prof. Dr. Stefan Leutenegger

Data Analytics and Machine Learning Group
Technische Universität München

Winter term 2024/2025

Section 1

Introduction

Another look at Logistic Regression

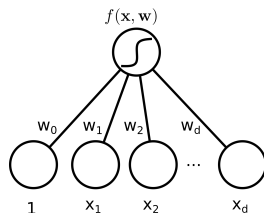
In [logistic regression](#) we model posterior distribution as:

$$y \mid \mathbf{x} \sim \text{Bernoulli}\left(\sigma(\mathbf{w}^T \mathbf{x})\right)$$

with $\mathbf{w}^T \mathbf{x} := w_0 + w_1 x_1 + \dots + w_D x_D$, and $\sigma(a) := \frac{1}{1 + \exp(-a)}$.

We can represent this graphically:

- each node is a (scalar) input¹;
- multiply the input with the weight on the edge: $x_j w_j$;
- compute weighted sum of incoming edges:
 $a_0 = \sum_{j=0}^D x_j w_j$;
- apply (activation) function:
 $p(y = 1 \mid \mathbf{x}) = \sigma(a_0) = f(\mathbf{x}, \mathbf{w})$.



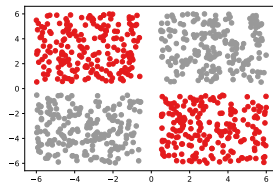
¹The first node $1 = x_0$ is for the bias term.

The XOR dataset

The XOR dataset is **not linearly separable**



Logistic regression will **fail** since it learns a linear decision boundary



How to handle non-linearity? Basis functions

We have input vectors $\mathbf{x}$ and associated targets y . We want to describe the underlying functional relation.

We can use the following simple model:

$$f(\mathbf{x}, \mathbf{w}) = \sigma \left(w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\mathbf{x}) \right) = \sigma(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})) \quad (1)$$

where

- ϕ **basis function** — many choices, can be nonlinear
- w_0 **bias** — equivalent to defining $\phi_0 \equiv 1$;
or adding constant 1 to every sample

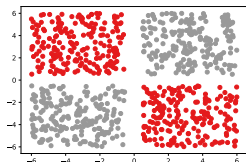
The basis function ϕ maps the samples to a new space where the data is linearly separable.

Remember we are **linear** in $\mathbf{w}$!

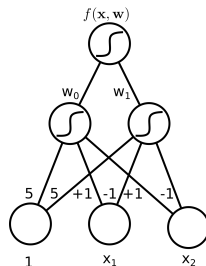
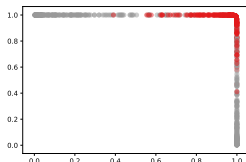
Example: Handling XOR dataset by custom basis functions

Apply a (nonlinear) transformation ϕ that maps samples to a space where they are linearly separable. For example:

$\mathbb{R}^2$ space



Transformed $\mathbb{R}^2$ space²



Here we defined a custom basis function $\phi : \mathbb{R}^3 \rightarrow \mathbb{R}^2$

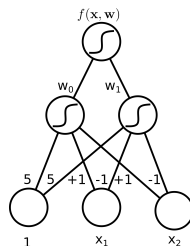
$$\phi(x) = \phi(1, x_1, x_2) = (\sigma(5 + x_1 + x_2) , \sigma(5 - x_1 - x_2))$$

²After applying σ .

Example: Handling XOR dataset by custom basis functions

The overall model is:

$$\begin{aligned} f(\mathbf{x}, \mathbf{w}) &= \sigma(\mathbf{w}^\top \boldsymbol{\phi}(\mathbf{x})) \\ &= \sigma_1 \left(\begin{bmatrix} w_0 & w_1 \end{bmatrix} \cdot \sigma_0 \left(\begin{bmatrix} 5 & 1 & 1 \\ 5 & -1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \right) \right) \end{aligned}$$



Q: How to find the parameters $\mathbf{w}$?

A: Train the model by minimizing a loss function.

This example is a *binary classification problem*, hence we minimize the *binary cross-entropy* loss:

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \sum_{n=1}^N - (y_n \log f(\mathbf{x}_n, \mathbf{w}) + (1 - y_n) \log (1 - f(\mathbf{x}_n, \mathbf{w})))$$

How to find the basis functions?

Different datasets require different ϕ to become (almost) linearly separable.

Idea: learn the basis function and the weights of the logistic regression *jointly* from the data (**end-to-end learning**).

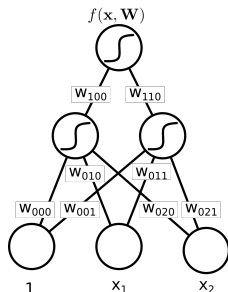
- *Previously:* only learn w_{100} and w_{110}
- *Now:* learn all w_{ijk}
where i =layer, j = input node, k = output node

$$f(\mathbf{x}, \mathbf{W}) = \sigma_1 \left([w_{100} \ w_{110}] \cdot \sigma_0 \left(\begin{bmatrix} w_{000} & w_{010} & w_{020} \\ w_{001} & w_{011} & w_{021} \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \right) \right)$$

$$\mathbf{W}^* = \arg \min_{\mathbf{W}} \sum_{n=1}^N - (y_n \log f(\mathbf{x}_n, \mathbf{W}) + (1 - y_n) \log (1 - f(\mathbf{x}_n, \mathbf{W})))$$

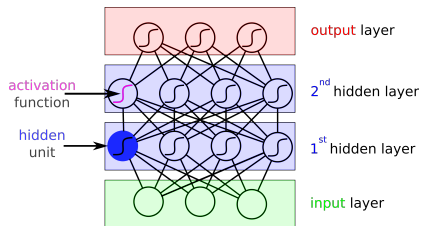
Feed-Forward **Neural Network** (FFNN) with **1 hidden** layer.

Note: σ_0 and σ_1 can be arbitrary activation functions.



Making the model more complicated

Each basis function can be a more complicated function of the feature vector x (a function of other basis functions rather than a function of x).



By adding more hidden layers we get a “**deep**” neural network³:

$$f(x, \mathbf{W}, \mathbf{b}) = \sigma_2 (\mathbf{W}_2 \sigma_1 (\mathbf{W}_1 \sigma_0 (\mathbf{W}_0 x + \mathbf{b}_0) + \mathbf{b}_1) + \mathbf{b}_2)$$

where $\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \mathbf{W}_2\}$, $\mathbf{b} = \{\mathbf{b}_0, \mathbf{b}_1, \mathbf{b}_2\}$ are the learnable **weights** and **biases**⁴.

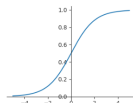
³We call this architecture a **Multi-layered Perceptron (MLP)**.

⁴Bias term explicitly added in the equation; not shown in the figure

Activation functions

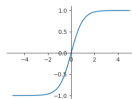
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



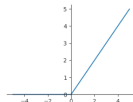
tanh

$$\tanh(x)$$



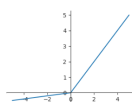
ReLU

$$\max(0, x)$$



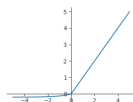
Leaky ReLU

$$\max(0.1x, x)$$



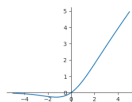
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Swish

$$x \cdot \sigma(x)$$



Most activation functions are applied **element-wise** when given a *multi-dimensional* input.

Softmax activation is a notable exception!

Why use nonlinear activation functions?

A function composed of multiple *linear* layers can be simplified:

$$\begin{aligned}f(\mathbf{x}, \mathbf{W}) &= (\mathbf{W}_L \ (\dots \ (\mathbf{W}_0 \mathbf{x}) \dots)) \\&= (\mathbf{W}_L \dots \mathbf{W}_1 \mathbf{W}_0) \mathbf{x} \\&= \mathbf{W}' \mathbf{x}\end{aligned}$$

hence, resulting in a *linear transformation*!

A function composed of multiple *nonlinear* layers cannot (in general) be simplified:

$$\begin{aligned}f(\mathbf{x}, \mathbf{W}) &= \sigma_L(\mathbf{W}_L \sigma_{L-1}(\dots \sigma_0(\mathbf{W}_0 \mathbf{x}) \dots)) \\&\neq \mathbf{W}' \mathbf{x}\end{aligned}$$

hence, we get a *nonlinear transformation*.

Non-linear transformations allows to learn more complex functions.

Neural networks are universal approximators

Universal approximation theorem

An MLP with a linear output layer and one hidden layer can approximate any continuous function defined over a closed and bounded subset of $\mathbb{R}^D$, under mild assumptions on the activation function ('squashing' activation functions; e.g. sigmoid) and given the number of hidden units is large enough.

[Cybenko 1989; Funahashi 1989; Hornik et al 1989, 1991; Hartman et al 1990].

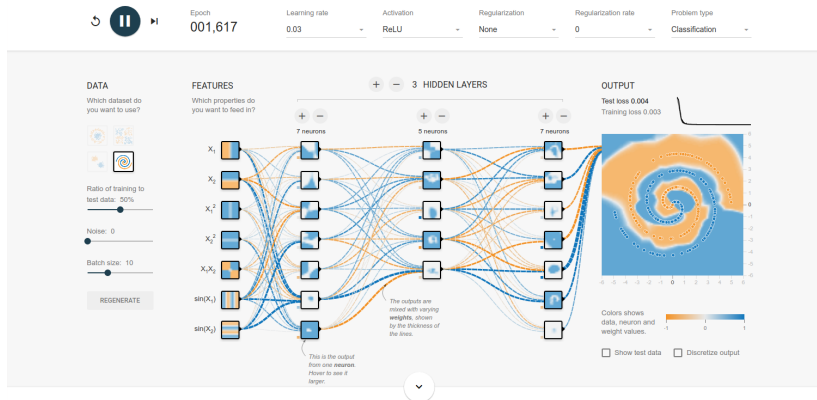
Also in the discrete case: a neural network can approximate any function from a discrete space to another.

Good news: Regardless of the function we aim to learn, there exists an MLP that can approximate that function arbitrarily well.

Bad news: Finding the MLP that provides the best approximation is generally a complex and tricky task.

Demo: <http://playground.tensorflow.org>

Don't Worry, You Can't Break It. We Promise.



Multiple hidden layers

According to the universal approximation theorem, a feed-forward network with 1 hidden layer can represent any function.

Q: Why adding more layers?

Theoretical reason:

- For some families of functions, if we use *a few* layers we would need a large number of hidden units (and therefore parameters). But we can get the same representation power by adding *more layers*, fewer hidden units, and **fewer parameters**.

Practical reason:

- Deeper networks (with some additional tricks) often **train faster** and **generalize better**.

Multiple hidden layers

Functions that can be compactly represented with k layers may require exponentially many hidden units when using only $k - 1$ layers.

Deep network can learn a **hierarchy of representations**.

Different high-level features share lower-level features.

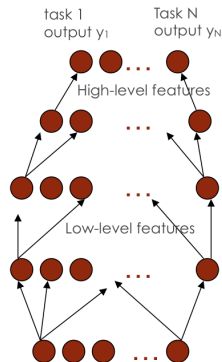


Figure from *Understanding and Improving Deep Learning Algorithms*, Yoshua Bengio, ML Google Distinguished Lecture, 2010

Section 2

Beyond binary classification

Loss function

Neural networks can be used for *various prediction tasks*.

Different tasks require changing

1. the activation function in the *final* layer
2. the loss function

For *supervised learning* common choices are

Prediction target	$p(y x)$	Final layer	Loss function
Binary	Bernoulli	Sigmoid	Binary cross entropy
Discrete	Categorical	Softmax	Cross entropy
Continuous	Gaussian	Identity	Squared error

Example 1: Binary classification

- Data: $\{\mathbf{x}_n, y_n\}_{n=1}^N$, where $y_n \in \{0, 1\}$.
- Activation in the final layer: **sigmoid**

$$f(\mathbf{x}, \mathbf{W}) = \frac{1}{1 + \exp(-a)}$$

where a is the output of the last layer before activation (*logits*).

- Conditional distribution: **Bernoulli**

$$p(y \mid \mathbf{x}) = \text{Bernoulli}(y \mid f(\mathbf{x}, \mathbf{W}))$$

- Loss function: **binary cross entropy**

$$\begin{aligned} E(\mathbf{W}) &= - \sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n) \\ &= - \sum_{n=1}^N \left(y_n \log f(\mathbf{x}_n, \mathbf{W}) + (1 - y_n) \log (1 - f(\mathbf{x}_n, \mathbf{W})) \right) \end{aligned}$$

Example 2: Multi-class classification

- Data: $\{\mathbf{x}_n, \mathbf{y}_n\}_{n=1}^N$, where $\mathbf{y}_n \in \{0, 1\}^K$ (one-hot notation)⁵.
- Activation in the final layer: **softmax**

$$f_k(\mathbf{x}, \mathbf{W}) = \frac{\exp(a_k)}{\sum_{j=1}^K \exp(a_j)}$$

- Conditional distribution: **categorical**

$$p(\mathbf{y} \mid \mathbf{x}) = \text{Categorical}(\mathbf{y} \mid f(\mathbf{x}, \mathbf{W}))$$

- Loss function: **categorical cross entropy**

$$\begin{aligned} E(\mathbf{W}) &= - \sum_{n=1}^N \log p(\mathbf{y}_n \mid \mathbf{x}_n) \\ &= - \sum_{n=1}^N \sum_{k=1}^K y_{nk} \log f_k(\mathbf{x}_n, \mathbf{W}) \end{aligned}$$

⁵ K is the number of classes.

Example 3: Single-output regression

- Data: $\{\mathbf{x}_n, y_n\}_{n=1}^N$, where $y_n \in \mathbb{R}$.
- Activation in the final layer: **identity** (no activation)

$$f(\mathbf{x}, \mathbf{W}) = a$$

- Conditional distribution: **Gaussian**

$$p(y \mid \mathbf{x}) = \mathcal{N}(y \mid f(\mathbf{x}, \mathbf{W}), 1)$$

- Loss function: **squared error** (a.k.a. Gaussian cross-entropy)

$$\begin{aligned} E(\mathbf{W}) &= - \sum_{n=1}^N \log p(y_n \mid \mathbf{x}_n) \\ &= \sum_{n=1}^N (y_n - f(\mathbf{x}_n, \mathbf{W}))^2 + \text{const.} \end{aligned}$$

Unsupervised deep learning

So far we have only considered neural networks for supervised learning.

Unsupervised deep learning is also a very active field of research, with models such as:

- **Autoencoder** (*covered later in this course*)
- **Variational autoencoder** (*covered in our DGM lecture*)
- **Generative adversarial networks** (GAN; *covered in our DGM lecture*)
- **Unsupervised representation learning** (e.g. word or node embeddings; *covered in our MLGS lecture*)

Choosing the loss

We are free to choose any loss that provides a **useful gradient for training**.

This choice includes both the function and which values to use and compare to. Many model types mainly differ in their choice of loss.

Example loss functions:

- **Cross entropy**: For supervised classification
- **Mean squared error (MSE)**
- **Mean absolute error (MAE)**: Useful if we have outliers, but gradient is independent of the distance from the optimum (advanced optimizers handle this surprisingly well, though).
- **Huber loss, LogCosh**: MSE at close distances, MAE far away. Combine benefits of MSE and MAE.
- **Wasserstein (earth mover's) distance, KL-divergence**: For continuous distributions

Section 3

Parameter learning

Minimizing the loss function

In practice $E(\mathbf{W})$ is often non-convex $\rightarrow$ optimization is tricky:

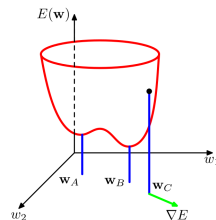
- a local minimum is not necessarily a global minimum;
- potentially there exist several local minima, many of which can be equivalent;
- often it is not possible to find a global minimum nor is it useful.

We may find a few local minima, and pick the one with higher performance on a validation set.

Default approach: Find a local minimum via gradient descent

- Start with some initial parameters $\mathbf{W}^{(0)}$
- Update the parameters in the direction of the negative gradient
- Repeat until convergence or stopping criterion is met

$$\mathbf{W}^{(new)} = \mathbf{W}^{(old)} - \tau \nabla_{\mathbf{W}} E(\mathbf{W}^{(old)})$$



How can we compute the gradient?

1. **By hand:** manually working out $\nabla_{\mathbf{W}} E$ and coding it is tricky and cumbersome (furthermore: see point 3).
2. **Numeric:** can be done as

$$\frac{\partial E}{\partial w_{ijk}} = \frac{E(w_{ijk} + \epsilon) - E(w_{ijk})}{\epsilon} + \mathcal{O}(\epsilon)$$

- each evaluation of the above equation roughly requires $\mathcal{O}(|\mathbf{W}|)$ operations, where $|\mathbf{W}|$ is the dimensionality of weight space;
- the evaluation has to be done for each parameter independently. Therefore computing $\nabla_{\mathbf{W}} E$ requires $\mathcal{O}(|\mathbf{W}|^2)$ operations!

How can we compute the gradient?

3. **Symbolic differentiation:** automates essentially how you would compute the gradient function by hand.
But: “writing down” the general expression for the gradient of every parameter is very expensive:
- potentially exponentially many different cases (e.g., when having multiple layers with ReLUs);
 - many terms reappear in the gradient computation for *different* parameters (since the function f is hierarchically constructed); these terms could be re-used to make computation faster; however symbolic differentiation does not exploit this insight.
4. **Automatic differentiation:** e.g., [backpropagation](#) for neural networks (see following slides):
- evaluate $\nabla_{\mathbf{W}} E(\mathbf{W})$ at the current point $\mathbf{W}$
 - evaluation in $\mathcal{O}(|\mathbf{W}|)$ (every “neuron” is visited only twice)

Backpropagation: Toy example

$$f(x) = \frac{2}{\sin(\exp(-x))}$$

$$f(x) = d(c(b(a(x))))$$

Modules:

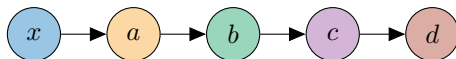
$$a(x) = -x$$

$$b(a) = \exp(a)$$

$$c(b) = \sin b$$

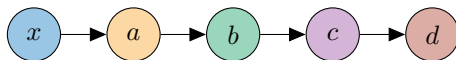
$$d(c) = \frac{2}{c}$$

Computational graph:



Backpropagation: Toy example

$$f(x) = \frac{2}{\sin(\exp(-x))}$$



Modules:

$$a(x) = -x$$

$$b(a) = \exp(a)$$

$$c(b) = \sin b$$

$$d(c) = \frac{2}{c}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial d}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial a} \frac{\partial a}{\partial x}$$

$\frac{\partial f}{\partial x}$ is the global derivative

$\frac{\partial d}{\partial c}$, $\frac{\partial c}{\partial b}$, $\frac{\partial b}{\partial a}$, $\frac{\partial a}{\partial x}$ are the local derivatives

Backpropagation in a nutshell

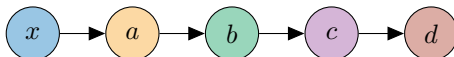
1. Write your function $f(x)$ as a composition of **modules**
 - the definition of a module depends on the specific task at hand
2. Work out the local derivative of each module **symbolically**
3. Do a **forward pass** for a given input x
 - i.e., compute $f(x)$ and remember the intermediate values for each module
4. Compute the **local derivatives** for x
5. Obtain the **global derivative** by multiplying the local derivatives

Work out the local derivatives

Compute the local derivative of each module symbolically

$$\begin{array}{ll} a(x) = -x & \frac{\partial a}{\partial x} = -1 \\ b(a) = \exp(a) & \frac{\partial b}{\partial a} = \exp(a) \\ c(b) = \sin b & \frac{\partial c}{\partial b} = \cos b \\ d(c) = \frac{2}{c} & \frac{\partial d}{\partial c} = -\frac{2}{c^2} \end{array}$$

Computational graph:



Forward pass

We want to compute the derivative $\frac{\partial f}{\partial x}$ at $x = -4.499$

In the forward pass, we compute f by following the computational graph and cache (memorize) the intermediate values of a, b, c, d

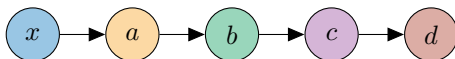
$$a = -x = 4.499$$

$$b = \exp(a) = 90$$

$$c = \sin b = 1$$

$$d = \frac{2}{c} = 2$$

Computational graph:



Backward pass

In the backward pass, we use the cached values of a, b, c, d to compute the local derivatives $\frac{\partial d}{\partial c}, \frac{\partial c}{\partial b}, \frac{\partial b}{\partial a}, \frac{\partial a}{\partial x}$

$$\frac{\partial a}{\partial x} = -1$$

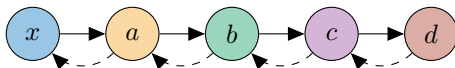
$$\frac{\partial b}{\partial a} = \exp(a) = \exp(4.499) = 90$$

$$\frac{\partial c}{\partial b} = \cos b = \cos 90 = 0$$

$$\frac{\partial d}{\partial c} = -\frac{2}{c^2} = -\frac{2}{1^2} = -2$$

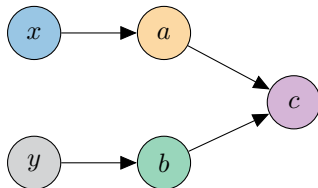
We obtain the global derivative by multiplying the local derivatives

$$\frac{\partial f}{\partial x} = \frac{\partial d}{\partial c} \frac{\partial c}{\partial b} \frac{\partial b}{\partial a} \frac{\partial a}{\partial x} = -2 \cdot 0 \cdot 90 \cdot -1 = 0$$



Multiple inputs

What if a function has multiple inputs?



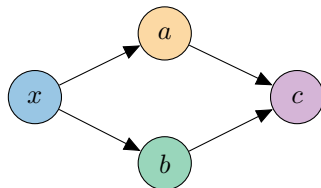
We compute the derivative along each of the paths

$$\frac{\partial c}{\partial x} = \frac{\partial c}{\partial a} \frac{\partial a}{\partial x}$$

$$\frac{\partial c}{\partial y} = \frac{\partial c}{\partial b} \frac{\partial b}{\partial y}$$

Multiple paths

What if a computational graph contains multiple paths from x to c ?

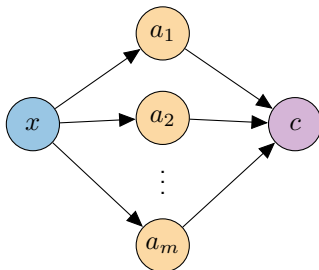


We need to sum the derivatives along each of the paths

$$\frac{\partial c}{\partial x} = \frac{\partial c}{\partial a} \frac{\partial a}{\partial x} + \frac{\partial c}{\partial b} \frac{\partial b}{\partial x}$$

Multivariate chain rule

The generalization to an arbitrary number of paths is given by the **multivariate chain rule**.



We need to sum the derivatives along all the paths

$$\frac{\partial c}{\partial x} = \sum_{i=1}^m \frac{\partial c}{\partial a_i} \frac{\partial a_i}{\partial x}$$

Jacobian and the gradient

Consider a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, and let $\mathbf{a} = f(\mathbf{x})$. The **Jacobian** is an $m \times n$ **matrix of partial derivatives**:

$$\frac{\partial \mathbf{a}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial a_1}{\partial x_1} & \cdots & \frac{\partial a_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial a_m}{\partial x_1} & \cdots & \frac{\partial a_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Let $g : \mathbb{R}^m \rightarrow \mathbb{R}$ and let $c = g(\mathbf{a})$.

The gradient $\nabla_{\mathbf{a}} c \in \mathbb{R}^m$ is the *transpose* of the Jacobian $\frac{\partial c}{\partial \mathbf{a}} \in \mathbb{R}^{1 \times m}$

$$\nabla_{\mathbf{a}} c = \left(\frac{\partial c}{\partial \mathbf{a}} \right)^T = \left[\frac{\partial c}{\partial a_1} \quad \cdots \quad \frac{\partial c}{\partial a_m} \right]^T$$

Note that the gradient $\nabla_{\mathbf{a}} c$ has the same shape as $\mathbf{a}$.

We are using the so-called numerator notation. Some other resources use a different (denominator) notation — be careful when using other books or slides.

Chain rule in matrix form

We can compactly represent the chain rule using Jacobian matrices

$$\frac{\partial c}{\partial x_j} = \sum_{i=1}^m \frac{\partial c}{\partial a_i} \frac{\partial a_i}{\partial x_j}$$

We can write this in **matrix form**:

$$\underbrace{\frac{\partial c}{\partial \mathbf{x}}}_{\in \mathbb{R}^{1 \times n}} = \underbrace{\frac{\partial c}{\partial \mathbf{a}}}_{\in \mathbb{R}^{1 \times m}} \underbrace{\frac{\partial \mathbf{a}}{\partial \mathbf{x}}}_{\in \mathbb{R}^{m \times n}}, \quad \text{where } \left[\frac{\partial \mathbf{a}}{\partial \mathbf{x}} \right]_{ij} = \frac{\partial a_i}{\partial x_j} \in \mathbb{R}$$

or equivalently:

$$\underbrace{\nabla_{\mathbf{x}} c}_{\in \mathbb{R}^n} = \underbrace{\left(\frac{\partial \mathbf{a}}{\partial \mathbf{x}} \right)^T}_{\in \mathbb{R}^{n \times m}} \underbrace{\nabla_{\mathbf{a}} c}_{\in \mathbb{R}^m}$$

Computational graph of a neural network

Consider a simple regression problem:

- Data: $\mathbf{x} \in \mathbb{R}^D, y \in \mathbb{R}$.
- Loss function: square error

Generate predictions with a feed-forward NN

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{h} = \sigma(\mathbf{a})$$

$$\hat{y} = \mathbf{V}\mathbf{h} + c$$

$$E = (\hat{y} - y)^2$$

In order to optimize $\mathbf{W}$ with gradient descent, we need to compute $\frac{\partial E}{\partial \mathbf{W}}$.

Computational graph of a neural network

Consider a simple regression problem:

- Data: $\mathbf{x} \in \mathbb{R}^D, y \in \mathbb{R}$.
- Loss function: square error

Generate predictions with a feed-forward NN

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$$\mathbf{h} = \sigma(\mathbf{a})$$

$$\hat{y} = \mathbf{V}\mathbf{h} + c$$

$$E = (\hat{y} - y)^2$$

In order to optimize $\mathbf{W}$ with gradient descent, we need to compute $\frac{\partial E}{\partial \mathbf{W}}$.
Does chain rule even make sense in this scenario?

$$\frac{\partial E}{\partial \mathbf{W}} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{W}}$$

What does it mean to take a derivative of a vector $\mathbf{a}$ w.r.t. a matrix $\mathbf{W}$?

Digression: matrix calculus

Consider a function having a specific input and output shape (e.g., scalar $\rightarrow$ scalar, vector $\rightarrow$ scalar, matrix $\rightarrow$ scalar, etc.).

		<i>Input is a ...</i>		
		scalar	vector	matrix
<i>Output is a ...</i>	scalar	scalar	vector	matrix
	vector	vector	matrix	3-way tensor
	matrix	matrix	3-way tensor	4-way tensor

Given $\mathbf{a} \in \mathbb{R}^H$ and $\mathbf{W} \in \mathbb{R}^{H \times D}$, $\frac{\partial \mathbf{a}}{\partial \mathbf{W}} \in \mathbb{R}^{H \times H \times D}$ is a 3-way tensor s.t.:

$$\left(\frac{\partial \mathbf{a}}{\partial \mathbf{W}} \right)_{ijk} = \frac{\partial a_i}{\partial W_{jk}}$$

Accumulating the gradient

The **local derivatives** (i.e. Jacobians), such as $\frac{\partial \mathbf{a}}{\partial \mathbf{W}}$, are large multidimensional tensors.

Materializing (computing) them is *expensive* and unnecessary.

In reality, we only care about the **global derivative**!



Assume that we know $\frac{\partial E}{\partial \mathbf{y}}$, and want to compute $\frac{\partial E}{\partial \mathbf{x}}$.

Most of the time, the Jacobian-vector product

$$\frac{\partial E}{\partial \mathbf{x}} = \frac{\partial E}{\partial \mathbf{y}} \frac{\partial \mathbf{y}}{\partial \mathbf{x}}$$

can be computed efficiently without materializing the Jacobian $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$!

Accumulating the gradient: Implementation

Backpropagation is typically implemented using the following abstraction.



Each module in the computational graph defines two functions

`forward`(x) given input x , compute output y

`backward` $\left(\frac{\partial E}{\partial y}\right)$ given the incoming global derivative $\frac{\partial E}{\partial y}$
compute the product $\frac{\partial E}{\partial x} = \frac{\partial E}{\partial y} \frac{\partial y}{\partial x}$

Then, $\frac{\partial E}{\partial x}$ is passed as input to the parent node in the computational graph, etc.

Example: Affine layer

Back to our problem of finding

$$\frac{\partial E}{\partial \mathbf{W}} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{W}}$$

An **affine layer** is defined as

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

where $\mathbf{x} \in \mathbb{R}^D$, $\mathbf{W} \in \mathbb{R}^{H \times D}$, $\mathbf{b} \in \mathbb{R}^H$, $\mathbf{a} \in \mathbb{R}^H$

forward($\mathbf{W}, \mathbf{x}, \mathbf{b}$): compute $\mathbf{W}\mathbf{x} + \mathbf{b}$

backward $\left(\frac{\partial E}{\partial \mathbf{a}}\right)$: compute

$$\frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{W}},$$

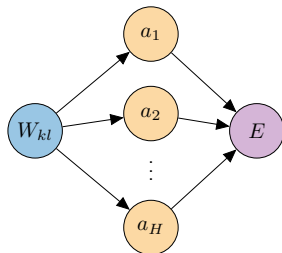
$$\frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{x}},$$

$$\frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{b}}$$

How can we implement this efficiently?

Step 1: work out the scalar derivative

First, let's find $\frac{\partial E}{\partial W_{kl}}$ for some k, l



$$\begin{aligned}\frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial W_{kl}} &= \sum_i \frac{\partial E}{\partial a_i} \frac{\partial a_i}{\partial W_{kl}} \\&= \sum_i \frac{\partial E}{\partial a_i} \frac{\partial (\mathbf{W} \mathbf{x} + \mathbf{b})_i}{\partial W_{kl}} \\&= \sum_i \frac{\partial E}{\partial a_i} \frac{\partial (\mathbf{W}_{i:} \mathbf{x})}{\partial W_{kl}} \\&= \sum_i \frac{\partial E}{\partial a_i} \frac{\partial (\sum_j W_{ij} x_j)}{\partial W_{kl}} \\&= \sum_i \sum_j \frac{\partial E}{\partial a_i} \frac{\partial W_{ij} x_j}{\partial W_{kl}} = \frac{\partial E}{\partial a_k} x_l\end{aligned}$$

Step 2: backward pass in matrix form

For each element W_{kl} we have

$$\frac{\partial E}{\partial W_{kl}} = \frac{\partial E}{\partial a_k} x_l$$

We can compute $\frac{\partial E}{\partial W_{kl}}$ for all elements k, l in matrix form

$$\times \begin{bmatrix} x_1 & \cdots & x_D \end{bmatrix}$$

$$\begin{bmatrix} \frac{\partial E}{\partial a_1} \\ \vdots \\ \frac{\partial E}{\partial a_H} \end{bmatrix} \begin{bmatrix} \frac{\partial E}{\partial W_{11}} & \cdots & \frac{\partial E}{\partial W_{1D}} \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial W_{H1}} & \cdots & \frac{\partial E}{\partial W_{HD}} \end{bmatrix}$$

Or more compactly

$$\frac{\partial E}{\partial \mathbf{W}} = \left(\frac{\partial E}{\partial \mathbf{a}} \right)^T \mathbf{x}^T$$

Matrix operations are much more efficient than for-loops thanks to [vectorization](#) (especially on GPUs)

Example: Affine layer

An affine layer is defined as

$$\mathbf{a} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

`forward`($\mathbf{W}, \mathbf{x}, \mathbf{b}$): compute $\mathbf{W}\mathbf{x} + \mathbf{b}$

`backward` $\left(\frac{\partial E}{\partial \mathbf{a}}\right)$: compute

$$\begin{aligned}\frac{\partial E}{\partial \mathbf{W}} &= \frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{W}} = \left(\mathbf{x} \frac{\partial E}{\partial \mathbf{a}}\right)^T \\ \frac{\partial E}{\partial \mathbf{x}} &= \frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{x}} = \frac{\partial E}{\partial \mathbf{a}} \mathbf{W} \\ \frac{\partial E}{\partial \mathbf{b}} &= \frac{\partial E}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{b}} = \frac{\partial E}{\partial \mathbf{a}}\end{aligned}$$

The derivatives for $\mathbf{x}$ and $\mathbf{b}$ are obtained similarly to $\mathbf{W}$.

Backpropagation: Summary

- Define the computation as a composition of modules
- The forward function computes the output of the module
- The backward function accumulates the total gradient
- We can implement backward without materializing the Jacobian
- Backpropagation walks backward through the computational graph, accumulating the product of gradients

Reading material

- Goodfellow, Deep Learning: Chapter 6

Acknowledgements

- Some slides based on an earlier version by Patrick van der Smagt
- Backpropagation slides are based on the materials from mlvu.github.io by Peter Bloem